FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING
Department of Computer Engineering

Course , Subject & Experiment Details

Academic Year	**2025-26**	**Estimated Time**	**02 - Hours**
Course & Semester	**T.E. (CMPN)- Sem V**	**Subject Name & Code**	**CSS- 25PCC13CE19**
Module No.	**03 – Mapped to CO- 3**	**Chapter Title**	**Cryptographic Hash Functions**

Practical No:	
Title:	**Performance Analysis of Hash Algorithms**
Date of Performance:	
Date of Submission:	
Roll No:	
Name of the Student:	

Evaluation:

Sr. No	Rubric	Grade
1	On time submission Or completion (2)	
2	Preparedness(2)	
3	Skill (4)	
4	Output (2)	

Signature of the Teacher:

Date:

Title: For varying message sizes, test integrity of message using MD-5, SHA-1, and analyse the performance of the two protocols.

Lab Objective :

This lab provides insight into:

- The working of MD5 and SHA-1 and variations of SHA-1 and analyze the performance of both for varying message sizes.

Reference : "Cryptography and Network Security" B. A. Forouzan
"Cryptography and Network Security" Atul Kahate
www.md5summer.org/download.html

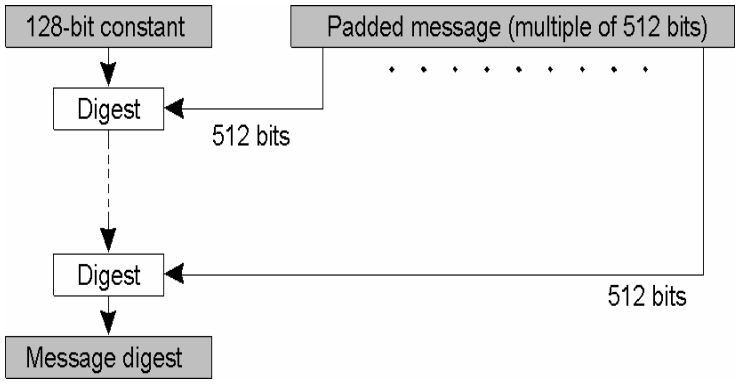
Prerequisite: Java or Python and Knowledge of hashing and Crypt API.

Theory:

Cryptographic hash functions are a very useful tool in cryptography. They are applied in many areas like integrity of messages, storage of passwords securely and protect signatures. The three hash algorithms SHA-1, SHA-512 and MD5 are considered to analyze their performance.

MD5

- Takes as input a message of arbitrary length and produces as output a 128 bit "fingerprint" or "message digest" of the input.
- It is conjectured that it is computationally infeasible to produce two messages having the same message digest.
- Intended where a large file must be "compressed" in a secure manner before being encrypted with a private key under a public-key cryptosystem such as PGP



Input:
Suppose a b-bit message as input, and that we need to find its message digest.

Algorithm:

Step 1 – append padding bits:

– The message is padded so that its length is congruent to 448, modulo 512.
- Means extended to just 64 bits of being of 512 bits long.
– A single "1" bit is appended to the message, and then "0" bits are appended so that the length in bits equals 448 modulo 512.

• Step 2 – append length

– A 64 bit binary representation of b is appended to the result of the previous step.
– The resulting message has a length that is an exact multiple of 512 bits.

• Step 3 – Divide the input into 512-bit blocks

Now we divide the input mesg into into blocks , each of length 512 bits.

• Step 4 – Initialize MD Buffer

- A four-word buffer (A,B,C,D) is used to compute the message digest.
– Here each of A,B,C,D, is a 32 bit register.
- These registers are initialized to the following values in hexadecimal:

word A: 01 23 45 67
word B: 89 ab cd ef
word C: fe dc ba 98
word D: 76 54 32 10

Four auxiliary functions

In addition MD5 uses four auxiliary functions that each take as input three 32-bit words and produce as output one 32-bit word. They apply the logical operators and, or, not and xor to the input bits.

Round 1 = (b and c) or ((not(b) and d))
Round 2 = (b and d) or (c and not(d))
Round 3 = B xor c xor d
Round 4 = C xor (b or not(d))

The Constant t[i] or k[i]

MD5 further uses a table K that has 64 elements. Element number i is indicated as K_i. The table is computed beforehand to speed up the computations. The elements are computed using the mathematical sin function:

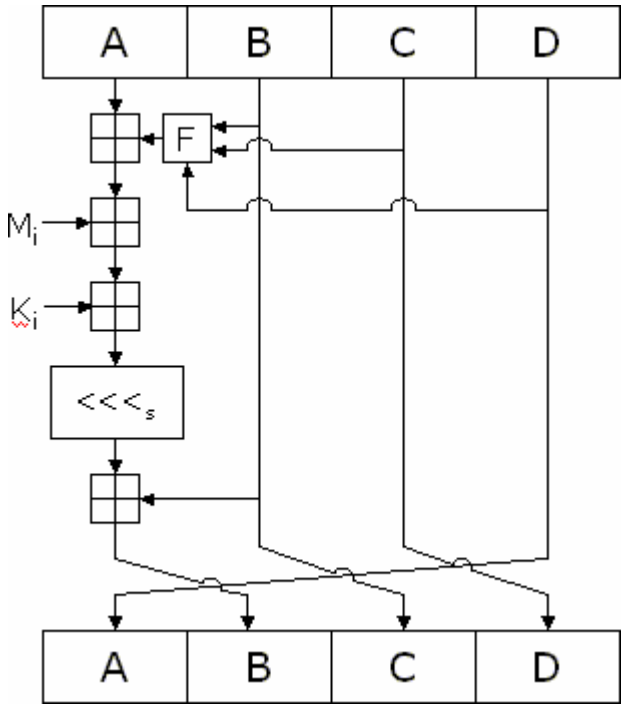
$K_i = abs(sin(i + 1)) * 2^{32}$

• Step 5 – Process message in 16-word blocks.

1. – Process message in 16-word (512-bit) blocks:

- Using 4 rounds of 16 bit operations on message block & buffer
- Add output to buffer input to form new buffer value

2. Output hash value is the final buffer value
3. The contents of the four buffers (A, B, C and D) are now mixed with the words of the input, using the four auxiliary functions (F). There are four *rounds*, each involves 16 basic *operations*. One operation is illustrated in the figure below.



The figure shows how the auxiliary function F is applied to the four buffers (A, B, C and D), using message word M_i and constant K_i. The item "<<<s" denotes a binary left shift by *s* bits.

Round 1.
[abcd k s i] denote the operation a = b + ((a + F (b, c, d) + X [k] + T [i]) <<< s).

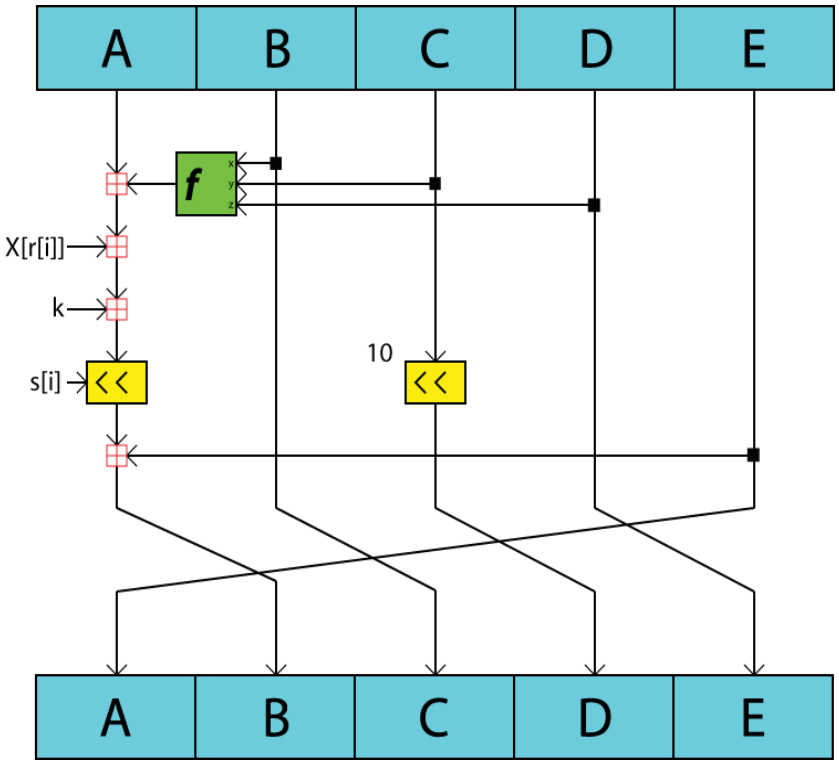
Do the following 16 operations.
[ABCD 0 7 1] [DABC 1 12 2] [CDAB 2 17 3] [BCDA 3 22 4]
[ABCD 4 7 5] [DABC 5 12 6] [CDAB 6 17 7] [BCDA 7 22 8]
[ABCD 8 7 9] [DABC 9 12 10] [CDAB 10 17 11] [BCDA 11 22 12]
[ABCD 12 7 13] [DABC 13 12 14] [CDAB 14 17 15] [BCDA 15 22 16]

Output:

– The message digest produced as output is A, B, C, D.
– That is, output begins with the low-order byte of A, and end with the high-order byte of D.

SHA-1

Processing is similar to SHA-1 with small variations. In SHA-1, chaining variables are 5 and Boolean operations are different.



Analysis

Differences between MD5 and SHA Algorithms

Keys For Comparison	**MD5**	**SHA**
Security	Less Secure than SHA	High Secure than MD5
Message Digest Length	128 Bits	160 Bits
Attacks required to find out original Message	2128 bit operations required to break	2160 bit operations required to break
Attacks to try and find two messages producing the	264 bit operations required to break	280 bit operations required to break

same MD		
Speed	Faster, only 64 iterations	Slower than MD5, Required 80 Iterations
Successful attacks so far	Attacks reported to some extents	No such attach report yet

MD5 Execution

Test Strings	MD5	SHA-1
1234567890	f807f1fcf80d030febe008fa1708e1ef31	
abcdefghijklm nopqrstuvwxyz	f3fcf3f711e2f4001dfb191cfa17f10b15	
message digest	f91b191d1ce7e3ed121a0f01eaf111f015	

Timing comparison between MD5 and SHA-1

File Size	MD5	SHA-1
1 KB		
5 KB		
10 KB		

Practical and Real Time Applications

- In Windows OS, PowerShell function "Get-FileHash"
- Android ROMs
- File servers - file servers often provide a pre-computed MD5 (known as md5sum) checksum for the files, so that a user can compare the checksum of the downloaded file to it.
- Most unix-based operating systems include MD5 sum utilities in their distribution packages

Conclusion:

The program was tested for different sets of inputs.
Program is working SATISFACTORY NOT SATISFACTORY(
Tick appropriate outcome)

Post Lab Assignment:

1. Why is SHA-1 more secure than MD5?
2. **Message Integrity Checker**
   Develop a simple application that works like a file-integrity checker:

   - Select a file.
   - Generate and store its MD5 and SHA-1 hashes.
   - Later select the same file.
   - Recalculate the hashes.
   - Compare the old and new values.
   - Display:
     File is unchanged, or **File has been modified**.
3. **Hash Collision Demonstration:**
   Implement a program that accepts multiple messages as input and generates their MD5 and SHA-1 hash values. Store the hash values and check whether any two different messages produce the same hash value. Display the messages and corresponding hash values, and explain the concept of a **hash collision** and its impact on message integrity.